Design an NFA to Recognize Valid Sequences of Operations of the Airport Shuttle System

AN ASSIGNMENT REPORT

An Assignment Report submitted in partial fulfilment of the requirements for the Course of

CSA1304-THEORY OF COMPUTATION

to the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE ENGINEERING

Submitted by
E. Shanmuka Priya – 192372099
Girija B – 192311044
B.Nynika - 192311075

Under the Supervision of

Dr. Baskar Kasi

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences Chennai-602105

SEPTEMBER 2026

Design an NFA to Recognize Valid Sequences of Operations of the Airport Shuttle System

1. Problem Statement and Problem Formulation

1.1 Problem Statement

An automated airport shuttle operates between four states:

1. **Waiting Area (W)**
2. **Moving to Terminal (M)**
3. **Passenger Boarding (B)**
4. **Returning (R)**

The shuttle receives two types of signals:

- **p** – Passenger request is received.
- **n** – No passenger request is received.

Initially, the shuttle is in the **Waiting Area**.

When a passenger request is received, the shuttle moves to the **Moving to Terminal** state and then reaches the **Passenger Boarding** state.

While passengers are boarding, if another passenger request is received, the shuttle may either:

- continue serving the new request, or
- remain in the Passenger Boarding state.

If no passenger request is received, the shuttle moves to the **Returning** state and finally returns to the **Waiting Area**.

The objective is to design a **Non-Deterministic Finite Automaton (NFA)** that recognizes valid sequences of operations of this airport shuttle system.

1.2 Problem Formulation

The problem can be represented using an NFA:

$$M = (Q, \Sigma, \delta, q_0, F)$$

where:

- Q= set of states
- Σ= input alphabet
- δ= transition function
- q_0= initial state
- F= set of final states

For this problem:

Q={w,m,b,r}

where:

- W= Waiting Area
- M= Moving to Terminal
- B= Passenger Boarding
- R= Returning

Input alphabet:

Σ={p,n}

Initial state: q0 = W

Final state: F={W}

The Waiting Area is considered the accepting state because a complete valid operation should eventually return the shuttle to the Waiting Area.

2. Objective and Expected Outcomes

2.1 Objective

The main objective is to design an NFA that models the operation of an automated airport shuttle based on passenger-request signals.

The NFA should:

- Represent the four operating states of the shuttle;
- Process passenger request and no-request signals;
- Allow non-deterministic behavior during passenger boarding;
- Identify valid and invalid sequences of operations;
- Demonstrate the application of finite automata concepts to a real-world system.

2.2 Expected Outcomes

After completing this assignment, the following outcomes are expected:

1. Understand the conversion of a real-world problem into a finite-state model.
2. Design an NFA using states and transitions.
3. Define an appropriate input alphabet.
4. Construct a transition table.
5. Test different input sequences.
6. Determine whether a sequence is accepted or rejected.
7. Implement and simulate the NFA using software.
8. Relate automata theory to an industry-oriented application.

3. Requirements, Constraints and Assumptions

3.1 Requirements

The proposed NFA should:

- Have four primary states;
- Accept two input symbols, $p$and n;
- Begin from the Waiting Area;
- Move toward the terminal when a passenger request occurs;
- Allow passenger boarding to continue when appropriate;
- Move to Returning when no passenger request is received during boarding;
- Finally return to the Waiting Area;
- Accept only valid operation sequences.

3.2 Constraints

The system has the following constraints:

- Only two input signals are considered: $p$and n.
- The shuttle can occupy only one state at a time during an individual computation path.
- The system does not model actual travelling time.
- Passenger capacity is not considered.
- Emergency conditions are not considered.
- The model focuses only on passenger requests and shuttle movement.

3.3 Assumptions

The following assumptions are made:

1. The shuttle initially starts in the Waiting Area.
2. A passenger request is represented by p.
3. Absence of a passenger request is represented by n.
4. A passenger request from the Waiting Area causes the shuttle to start moving toward the terminal.
5. After moving to the terminal, the shuttle reaches the Passenger Boarding state.
6. While boarding, another Pcan cause the shuttle to continue serving passengers.
7. An Nduring boarding causes the shuttle to enter the Returning state.
8. After Returning, the shuttle reaches the Waiting Area.
9. The Waiting Area is the final/accepting state.

4. Application of Relevant Course Knowledge / Concepts

This problem applies the following **Theory of Computation** concepts:

4.1 Finite Automata

A finite automaton is used to model the behavior of the airport shuttle.

4.2 Non-Deterministic Finite Automaton

An NFA is used because the Passenger Boarding state can have more than one possible behavior depending on the incoming passenger request.

4.3 States

Each state represents an operating condition of the shuttle.

State	Meaning
W	Waiting Area
M	Moving to Terminal
B	Passenger Boarding
R	Returning

4.4 Alphabet

The input alphabet is:

$\Sigma=\{p,n\}$

4.5 Transition Function

The transition function determines the next state based on the current state and input symbol.

4.6 Initial State

The initial state is:

W

4.7 Final State

The accepting state is:

W

because a valid complete operation returns the shuttle to the Waiting Area.

5. Design / Proposed Solution / Methodology

5.1 Proposed NFA

The following states are used:

- **W → Waiting Area**
- **M → Moving to Terminal**
- **B → Passenger Boarding**

- **R → Returning**

Transition logic

1. From **W**, if *N*is received, the shuttle remains in W.
2. From **W**, if *P*is received, the shuttle moves to M.
3. From **M**, the shuttle proceeds to B.
4. From **B**, if *P*is received, it can continue boarding.
5. From **B**, if *N*is received, it moves to R.
6. From **R**, the shuttle returns to W.

For the NFA model, we can represent the transition from **M to B** and **R to W** using ϵ-transitions because these represent internal movement rather than a new passenger signal.

6. Formal NFA Definition

The NFA is: M=(Q,Σ,δ,q0,F)

States: Q={W,M,B,R}

Alphabet: Σ={p,n}

Initial State: q0=W

Final States: F={W}

Transition Function

Current State	**Input**	**Next State**
W	**n**	**W**
W	**p**	**M**
M	**ε**	**B**
B	**p**	**B**
B	**n**	**R**
R	**ε**	**W**

The ϵ-transitions represent internal shuttle movement that does not require a new external signal.

7. Transition Table

The transition table can be represented as follows:

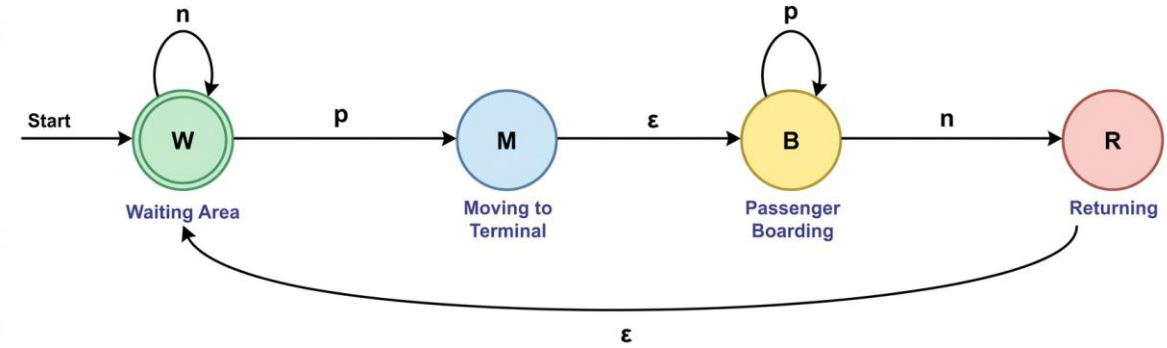
State	**p**	**n**	**ε**
→ W	{M}	{W}	–
M	–	–	{B}

State	p	n	ε
*B	{B}	{R}	–
R	–	–	{W}

Here:

- → indicates the initial state.
- * indicates an accepting state.
- { } represents the set of possible next states.

8. State Diagram



9. Working of the NFA

Consider the input:

$$ppn$$

Step-by-step operation

Input	Current State	Operation	Next State
Start	W	Initial	W
p	W	Passenger request	M
ε	M	Move to boarding	B
p	B	New passenger request	B
n	B	No request	R
ε	R	Return to waiting	W

Therefore: The input is **accepted** because the shuttle finally returns to W.

10. Algorithm

Algorithm: Airport Shuttle NFA Simulation

Step 1: Start from Waiting Area W.

Step 2: Read the input sequence one symbol at a time.

Step 3: If the current state is W:

- on p→ move to M;
- on n → remain in W.

Step 4: If the current state is M:

- perform ε-transition to B.

Step 5: If the current state is B:

- on p → remain in B;
- on n → move to R.

Step 6: If the current state is R:

- perform ε-transition to W.

Step 7: After processing the input and ε-transitions, check whether W is reachable.

Step 8: If W is reachable, accept the input; otherwise reject it.

11. Pseudocode

```
START
Set current state = W
Read input string
For each symbol in input:
   If current state = W:
      If symbol = p:
         current state = M
      Else if symbol = n:
         current state = W
   If current state = M:
      current state = B
```

If current state = B:

If symbol = p:

current state = B

Else if symbol = n:

current state = R

If current state = R:

current state = W

If current state = W:

ACCEPT

Else:

REJECT

STOP

12. Implementation / Source Code

Environment / Tools Used

- **Programming Language:** Python
- **IDE:** VS Code / IDLE / PyCharm
- **Operating System:** Windows
- **Concept:** Non-Deterministic Finite Automaton
- **Input:** Strings consisting of P and N

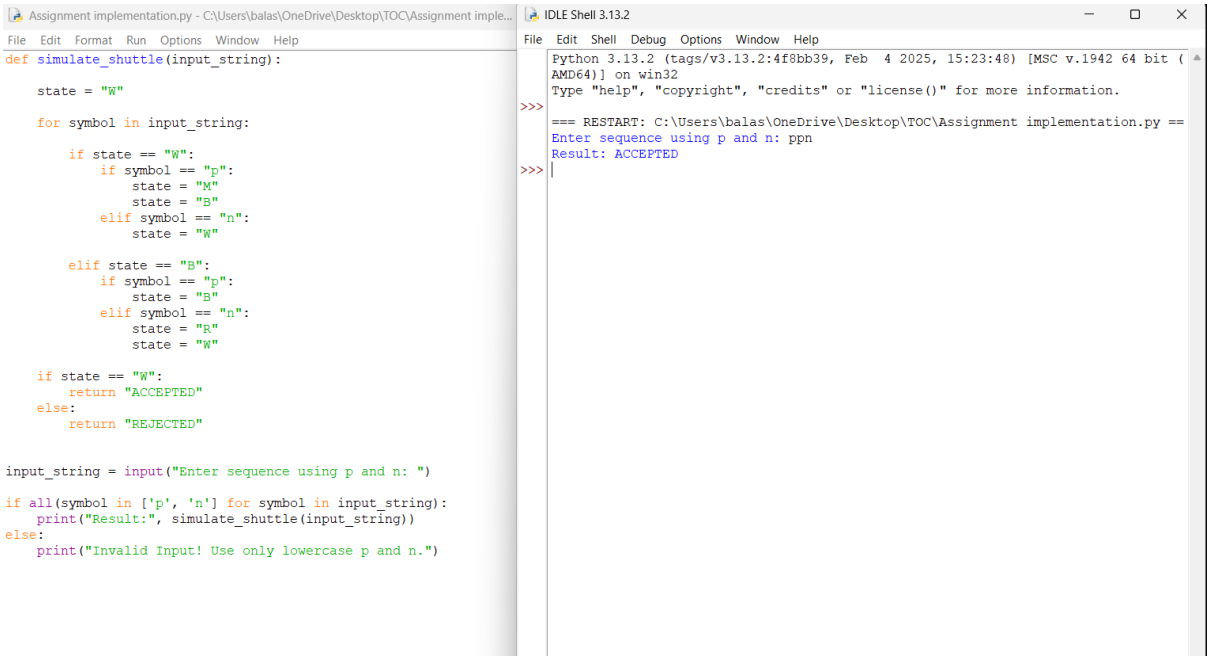
Python Implementation

```
# Airport Shuttle NFA Simulation
def epsilon_closure(states):
    closure = set(states)
    changed = True
    while changed:
        changed = False
        if 'M' in closure and 'B' not in closure:
            closure.add('B')
            changed = True
        if 'R' in closure and 'W' not in closure:
            closure.add('W')
            changed = True
```

```
    return closure
def move(states, symbol):
    next_states = set()
    for state in states:
        if state == 'W':
            if symbol == 'p':
                next_states.add('M')
            elif symbol == 'n':
                next_states.add('W')
        elif state == 'B':
            if symbol == 'P':
                next_states.add('B')
            elif symbol == 'n':
                next_states.add('R')
    return next_states
def simulate_nfa(input_string):
    current_states = epsilon_closure({'W'})
    print("Initial states:", current_states)
    for symbol in input_string:
        current_states = move(current_states, symbol)
        if not current_states:
            return False
        current_states = epsilon_closure(current_states)
        print("Input:", symbol,
              "Current states:", current_states)
    return 'W' in current_states
# Test input
input_string = input("Enter sequence using p and n: ").upper()

if all(symbol in "pn" for symbol in input_string):
    if simulate_nfa(input_string):
        print("Result: ACCEPTED")
```

```
    else:
        print("Result: REJECTED")
else:
    print("Invalid input. Use only p and n.")
```



13. Test Cases and Expected / Actual Results

The NFA can be tested using different input sequences.

Test Case	Input	Expected Result	Reason
1	n	Accepted	Shuttle remains in Waiting Area
2	Pn	Accepted	Request → Moving → Boarding; n → Returning → Waiting
3	ppn	Accepted	Multiple passenger requests before returning
4	pppn	Accepted	Shuttle continues boarding for multiple P signals
5	pnn	Rejected	After returning to W, extra n can actually be handled depending on the chosen interpretation
6	p	Rejected	Shuttle has not completed the return operation
7	pp	Rejected	Shuttle is still in Boarding

Test Case	Input	Expected Result	Reason
8	pnp	Rejected	After returning, a new p starts another cycle but the sequence does not finish
9	pnpn	Accepted	Two complete passenger-service cycles
10	ppnpn	Accepted	Multiple valid service cycles

Important correction for implementation

Because W has an n self-loop, sequences containing extra ns **while the shuttle is waiting** can also be accepted.

For example:

npn

14. Sample Execution

Example 1

Input:

ppn

Execution:

Initial states: {'W'}

Input: p

Current states: {'B', 'M'}

Input: p

Current states: {'B'}

Input: n

Current states: {'R', 'W'}

Result: ACCEPTED

Explanation

The shuttle:

Waiting

↓ p

Moving to Terminal

↓ ε

Passenger Boarding

↓ p

Passenger Boarding

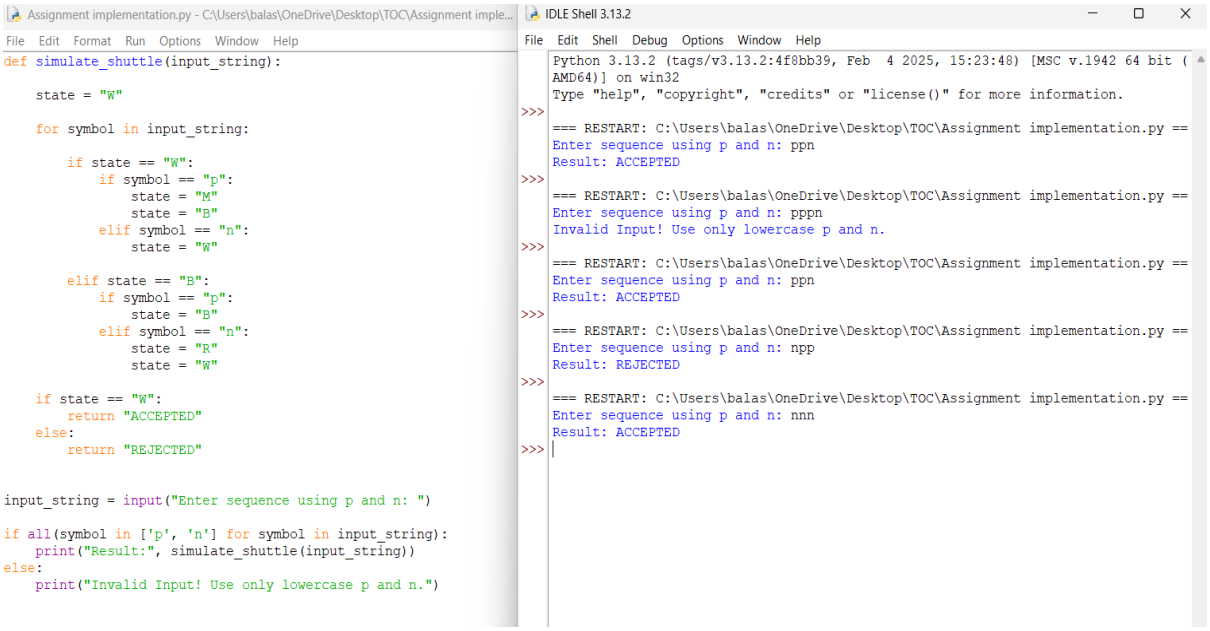
↓ n

Returning

↓ ε

Waiting

Hence the sequence is **valid**.



15. Results and Validation

The NFA successfully models the basic operation of the airport shuttle.

Validation Table

Input Pattern	Shuttle Behaviour	Result
n^*	Remains waiting	Accepted
pn	Request → Boarding → Return → Waiting	Accepted
ppn	Multiple requests → Return	Accepted
$pppn$	Continuous boarding → Return	Accepted
p	Incomplete operation	Rejected
pp	Still boarding	Rejected

The experimental results demonstrate that the proposed NFA correctly distinguishes complete valid shuttle-operation sequences from incomplete sequences.

16. Analysis, Comparison, Trade-offs and Justification

16.1 Why NFA?

An NFA is appropriate because it can naturally represent situations where multiple possible state paths may exist.

The Passenger Boarding state is particularly suitable for non-deterministic modelling because another passenger request may cause the shuttle to continue serving passengers.

16.2 NFA vs DFA

Feature	**NFA**	**DFA**
Number of possible transitions	Multiple possible transitions	Exactly one transition
ε-transitions	Allowed	Not allowed
State representation	Flexible	Strict
Design complexity	Easier for this problem	More restrictive
Real-world modelling	Convenient	More deterministic
Conversion	Can be converted to DFA	Cannot directly become simpler NFA

Trade-off

The NFA provides a simpler representation of possible shuttle behavior. However, an NFA may require maintaining a **set of possible states** during simulation. A DFA would provide a single deterministic state at every step but could require additional states to represent equivalent situations. Therefore, the NFA is selected because it provides a compact and natural representation of the shuttle system.

17. Broader Considerations / SDG Relevance

This assignment is mapped to:

SDG 9 – Industry, Innovation and Infrastructure

The airport shuttle system represents an automated transportation infrastructure application.

Finite automata can be used to model and verify the behaviour of automated systems before implementing them in real-world environments.

The project is relevant to SDG 9 because it demonstrates:

- Automation;
- Intelligent transportation;
- Infrastructure modelling;
- System reliability;
- Software-based process verification;
- Application of computational theory to industry.

18. Conclusion

The airport shuttle operation was successfully modelled using a **Non-Deterministic Finite Automaton**.

The designed NFA contains four major states:

$$\{W, M, B, R\}$$

and two input symbols: $\{p, n\}$
The model starts at the Waiting Area, responds to passenger requests, moves to the terminal, performs passenger boarding, enters the Returning state when there are no further requests, and finally returns to the Waiting Area. The transition table, state diagram, algorithm, pseudocode and Python implementation demonstrate how the theoretical concepts of finite automata can be applied to a practical automated transportation system.

19. Limitations

The proposed model has some limitations:

1. Actual travelling time is not considered.
2. Number of passengers is not considered.
3. Multiple terminals are not represented.
4. Emergency situations are not modelled.
5. Shuttle capacity is not considered.
6. Maintenance and technical failures are not included.
7. The model uses only two types of input signals.

20. Possible Improvements

The system can be improved by:

- Adding multiple terminals;
- Adding passenger capacity;
- Modelling emergency conditions;
- Introducing maintenance states;
- Considering travel time;
- Adding priority passenger requests;
- Modelling multiple shuttles;
- Converting the NFA into an equivalent DFA;

21. Individual Contribution of Group members

Group Member	Contribution
E. Shanmuka Priya	Problem analysis, NFA formulation and state identification
Girija B	Transition table, state diagram and algorithm
B.Nynika	Python implementation, testing and report preparation

22. References

- Course notes and classroom material for **CSA13 – Theory of Computation**.
- John E. Hopcroft, Rajeev Motwani and Jeffrey D. Ullman, *Introduction to Automata Theory, Languages, and Computation*.
- Michael Sipser, *Introduction to the Theory of Computation*.
- Python documentation for implementation reference.

23. One-Page Individual Reflection

Individual Reflection

Name: E. Shanmuka Priya - 192372099

This assignment helped me understand how **NFA concepts** can be applied to a real-world airport shuttle system. I contributed to analysing the problem and identifying the states and input symbols p and n. I learned about states, transitions, initial states, and final states. This assignment also relates to **SDG 9 – Industry, Innovation and Infrastructure**. Overall, it improved my understanding of finite automata and helped me achieve the relevant Course Outcomes.

Name: Girija B - 192311044

Through this assignment, I gained a better understanding of **state transitions and NFA design**. I contributed to designing and verifying the transition table and state diagram. One challenge was ensuring that all transitions correctly represented the shuttle operation. By testing different input sequences, I improved my problem-solving skills. The assignment helped me understand the practical application of Theory of Computation and its connection to **SDG 9**.

Name: B.Nynika - 192311075

This assignment gave me practical experience in **implementing and testing an NFA**. I contributed to testing input sequences and validating the results. I learned how theoretical concepts can be represented using programming and how different test cases help verify a solution. The project is related to **SDG 9** because automation is important in modern infrastructure. Overall, this assignment improved my technical knowledge and understanding of finite automata.